

lecture_23

September 26, 2026

1 Lecture 23: Metaheuristics

{note} Modules 1 and 2 built a toolkit that finds **provably optimal** solutions - the Simplex Algorithm for linear programs, gradient-based and interior-point methods for non-linear programs - provided the problem is continuous and, ideally, convex. Many transportation decisions are neither: the Vehicle Routing Problem's decision variables are **which sequence** to visit customers in, not **how much** flow to send, and the number of candidate sequences grows factorially with the number of customers. This lecture opens Module 3 by introducing metaheuristics - general-purpose search frameworks that trade the guarantee of optimality for the ability to handle exactly this kind of problem - and previews the three paradigms (local search, population search, swarm intelligence) this module builds in detail.

Learning Objectives

By the end of this notebook, you will be able to: 1. Explain why the combinatorial, dynamic, and stochastic nature of many transportation problems resists the exact and gradient-based methods of Modules 1-2, motivating the use of metaheuristics. 2. Distinguish local search, population search, and swarm intelligence as three metaheuristic paradigms, in terms of how many candidate solutions each maintains and how each balances exploration against exploitation. 3. Match transportation applications to the paradigm best suited to them. 4. Describe this module's roadmap and its three running examples — the Ackley function (introducing each algorithm), standard TSPLIB routing benchmarks (calibrating and stress-testing each algorithm at scale), and the Chennai Metro Feeder Route (the applied capacitated capstone).

Prerequisites: None beyond general familiarity with optimization problem formulation (Lectures 01-22) — this lecture is a self-contained conceptual bridge into Module 3.

Estimated time: 50 minutes

1.1 Why Metaheuristics?

The word metaheuristic combines the Greek prefix *meta* (“high-level” or “beyond”) with *heuristic*, from *heuriskein* (“to search” or “to find”). A metaheuristic is a high-level, general-purpose search strategy — not tied to any one problem — that guides an underlying search procedure to find good, though not necessarily provably optimal, solutions.

Modules 1 and 2 solved problems where two properties held: the decision variables were continuous (how much flow, what coordinates, what green-time split), and the objective/constraints were smooth enough for a gradient or a simplex pivot to make progress. Many transportation decisions violate both properties at once:

“{warning} Combinatorial decision space: A delivery company sequencing n stops faces $(n-1)!/2$ distinct routes — for just 15 stops, that is over 43 billion possibilities. There is no “gradient” of a route; swapping the order of two stops is a discrete jump, not an infinitesimal step.

Dynamic and stochastic uncertainty: Customer demand, travel times, and even which customers need service at all can vary hour to hour and day to day. A solution that is optimal for this morning’s data may be badly wrong by this afternoon. “

Three recurring examples illustrate this module’s stakes:

- Parking allocation: a city with n parking lots, each with limited capacity, wants an equitable, demand-responsive allocation policy — but demand is highly time- and weather-dependent, and the allocation itself is combinatorial.
- Transit timetabling: Chennai Metro Rail (CMRL) wants a daily train timetable across multiple routes that minimizes total passenger wait time — a combinatorial scheduling problem layered on top of demand that varies by time of day and day of week.
- Vehicle Routing Problem (VRP): a logistics company must sequence deliveries to n customers using a limited fleet — this module’s central application, calibrated and stress-tested on standard benchmarks starting Lecture 26, and applied to this course’s own Chennai Metro Feeder Route as the capstone in Lecture 33.

None of these problems has a usable gradient, and none can be solved to guaranteed optimality fast enough for practical fleet sizes. Metaheuristics accept this and instead search the solution space intelligently: they iteratively generate and evaluate candidate solutions, using two complementary mechanisms — exploration (diversification, searching new regions of the solution space) and exploitation (intensification, refining promising solutions already found) — to converge on a high-quality, though not certifiably optimal, solution within a practical amount of time.

1.2 Local Search, Population Search, and Swarm Intelligence

Metaheuristics differ primarily in *how many* candidate solutions they carry forward at once, and what mechanism drives exploration versus exploitation:

Paradigm	Carries forward	Exploration mechanism	Exploitation mechanism	Algorithm in this course
Local search	One solution at a time	Occasionally accept a worse neighbouring solution	Move to a better neighbouring solution	Simulated Annealing (Lectures 24-26)
Population search	A population of solutions	Recombine and mutate solutions to create new ones	Selection pressure favouring fitter solutions	Genetic Algorithm (Lectures 27-29)

Paradigm	Carries forward	Exploration mechanism	Exploitation mechanism	Algorithm in this course
Swarm intelligence	A colony of agents (ants), coordinated indirectly	Probabilistic construction, biased partly by randomness	Reinforcement of good choices via shared pheromone	Ant Colony Optimization (Lectures 30-32)

{note} All three paradigms share the same skeleton: start from an initial solution (or population, or colony), repeatedly generate new candidates and decide whether to keep them, track the best solution found, and stop once a convergence criterion is met. What differs is *how* new candidates are generated and *what* determines whether they replace what came before – precisely the two design choices each of this module's three algorithms will make differently.

Local search is the simplest and cheapest per iteration, but — carrying only one solution — is the most prone to getting trapped near a *local* optimum. Population and swarm methods pay a higher per-iteration cost (many candidates to evaluate) in exchange for a built-in defense against this: a population that hasn't converged still holds diverse solutions exploring different regions of the search space, while a colony's shared pheromone memory accumulates evidence across many ants' independent constructions rather than depending on any one of them.

1.3 Applications in Transportation Engineering

The choice of paradigm is rarely arbitrary — it follows from how the problem's uncertainty and decision structure interact:

Application	Local Search	Population Search	Swarm Intelligence
Parking Allocation			
Transit Timetabling			
Vehicle Routing Problem			
Traffic Signal Optimisation			
Vehicle Navigation			

{tip} The Vehicle Routing Problem appears in all three columns for a reason: it is combinatorial enough to reward population and swarm search's ability to hold multiple candidate routes at once, yet structured enough (a single tour, or a small set of vehicle routes) that local search's neighbourhood moves – such as swapping the order of two stops – remain cheap and effective. This is exactly why Lecture 33 uses the VRP to compare all three algorithms head-to-head, on equal footing.

1.4 Module Roadmap

Each of the three algorithms in this module is developed across an identical 3-lecture arc:

1. Motivation & Pseudocode (24, 27, 30): the intuition behind the algorithm, its formal pseudocode, and a small hand-traced example.
2. Algorithm (25, 28, 31): a working Python implementation, applied to a standard synthetic test function (the Ackley function) — deliberately the simplest possible setting, so the lecture can focus entirely on turning pseudocode into code, without a routing problem's added structure to work through at the same time.
3. Benchmarking (26, 29, 32): calibrating the algorithm's parameters on a small TSPLIB instance (eil51, 51 stops, published optimal 426) under a limited budget, to build intuition for how each parameter trades off exploration against exploitation — then asking the question a calibration study is ultimately for: does that calibration still hold on a substantially larger instance (eil101, 101 stops, published optimal 629)?

Lecture 33 closes the module with the Chennai Metro Feeder Route — this course's own network, unseen since this lecture's preview — escalated directly to a genuine Capacitated Vehicle Routing Problem, carrying the parameters calibrated in Lectures 26, 29, and 32 rather than tuning fresh ones, and comparing all three algorithms side by side on solution quality, convergence speed, and runtime.

{note} Simulated Annealing and the Genetic Algorithm are each implemented once, as a generic **engine** with **pluggable operators** (the problem-specific pieces - how a neighbour is generated, how two solutions recombine): the exact same ``sa()`` and ``ga()`` functions from Lectures 25 and 28 reappear unchanged in Lectures 26 and 29, first calibrating on ``eil51`` and then, with nothing but the neighbourhood/operators swapped, running on ``eil101`` and again on the Chennai Metro Feeder Route in Lecture 33. Ant Colony Optimization follows the same **principle** - construct candidates, reinforce what works - but takes two genuinely different forms: ACOR (Lecture 31), an archive-based bridge needed only because Ackley has no graph for pheromone to live on, and native graph-based ACO (introduced fresh in Lecture 32, once the module starts working with real routing benchmarks that **are** graphs). Because Lecture 32's calibration happens directly on ``eil51`` using native ACO, it transfers to ``eil101`` and to Chennai exactly the way SA's and GA's calibrations do - ACOR's role stays confined to bridging Ackley alone.

1.5 Circling Back

- Lectures 01-11 (Linear Programming) and Lectures 12-22 (Non-Linear Programming): both modules relied on the decision space being continuous (or, for LP, on extreme points of a polyhedron) and on the objective/constraints being smooth enough for the Simplex Algorithm or gradient-based methods to make guaranteed progress. This module's opening argument is precisely that many transportation problems — starting with the Vehicle Routing Problem — violate both assumptions, motivating a different toolkit entirely.

1.6 Moving Forward

- Lecture 24 (Simulated Annealing: Motivation & Pseudocode): the first of the three algorithms, and the simplest — a single-solution local search method inspired by the physical process of metal annealing.

1.7 Further Reading

- Blum, C. and Roli, A. (2003). “Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison.” *ACM Computing Surveys*, 35(3), 268-308.
- Glover, F.W. and Kochenberger, G.A. (Eds.) (2003). *Handbook of Metaheuristics*. Springer.
- Talbi, E-G. (2009). *Metaheuristics: From Design to Implementation*. Wiley.
- Reinelt, G. (1991). “TSPLIB — A Traveling Salesman Problem Library.” *ORSA Journal on Computing*, 3(4), 376-384.
- Toth, P. and Vigo, D. (Eds.) (2014). *Vehicle Routing: Problems, Methods, and Applications* (2nd ed.). SIAM — Chapter 1 (problem taxonomy and solution approaches).